



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 6

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Федуков А. А.

Преподаватель: Цалкович П.А.

12 мая 2025 г.

Цель работы

Целью данной лабораторной работы является построение реалистичного изображения трёхмерной сцены средствами библиотеки OpenGL. В рамках работы требуется:

- определить и настроить параметры модели освещения OpenGL, включая свойства источников света, материалы объектов сцены и характеристики глобальной модели освещения;
- исследовать влияние параметров компонентов модели освещения источника света (вариант А3) на итоговое изображение;
- реализовать анимацию движения тела при условии абсолютно упругого отражения от границ ограничивающего объёма (вариант Б2);
- реализовать наложение текстуры, определяющей интенсивность поверхности, с возможностью её отключения (вариант В1).

Работа базируется на лабораторной работе №3, включающей реализацию модельно-видовых и проекционных преобразований.

Теория

Модель освещения в OpenGL

OpenGL использует классическую модель освещения Фонга, включающую следующие компоненты:

- **Фоновое (ambient) освещение** — постоянная составляющая освещения, моделирующая рассеянный свет в окружающей среде.
- **Диффузное (diffuse) освещение** — моделирует рассеянное отражение света от матовой поверхности. Интенсивность зависит от угла между нормалью к поверхности и направлением на источник света.

- **Зеркальное (specular) освещение** — моделирует блики на блестящих поверхностях. Интенсивность зависит от угла между направлением отражения и направлением на наблюдателя.

Световой источник задаётся следующими параметрами: цвет и интенсивность каждой компоненты (ambient, diffuse, specular), положение в пространстве (точечный или направленный свет), а также дополнительные параметры (например, затухание и фокусировка, если используется).

Вариант А3: Влияние параметров источника света

Параметры источника света включают в себя:

- `GL_AMBIENT` — цвет и интенсивность фонового света от источника;
- `GL_DIFFUSE` — цвет и интенсивность рассеянного света;
- `GL_SPECULAR` — цвет и интенсивность зеркального света;
- `GL_POSITION` — положение источника (или направление, если используется бесконечно удалённый свет);
- `GL_CONSTANT_ATTENUATION`, `GL_LINEAR_ATTENUATION`, `GL_QUADRATIC_ATTENUATION` — коэффициенты ослабления интенсивности с расстоянием.

Изменение этих параметров влияет на реалистичность сцены: яркость, цветовое оформление, резкость бликов, распределение света по объектам.

Вариант Б2: Упругое отражение от границ объёма

Моделируется движение тела с заданной начальной скоростью в трёхмерном ограничивающем объёме (обычно кубе или прямоугольном параллелепипеде). При столкновении с границей компонента скорости по соответствующей оси инвертируется, имитируя абсолютно упругое отражение. Например, при столкновении со стенкой по оси x :

$$v_x \leftarrow -v_x$$

При этом положение объекта пересчитывается каждый кадр с учётом прошедшего времени Δt :

$$\vec{r}_{\text{new}} = \vec{r}_{\text{old}} + \vec{v} \cdot \Delta t$$

Вариант В1: Текстура как карта интенсивности

Использование текстуры для управления интенсивностью позволяет применять изображение (например, загруженное из BMP-файла) в качестве множителя освещённости. Это моделирует неровности, детали или узоры на поверхности без геометрической детализации. Текстурные координаты привязываются к вершинам, а при рендеринге производится выборка цвета из текстуры.

В OpenGL это достигается с помощью механизма наложения текстур и смешивания их с освещением (модуляция):

$$I_{\text{final}} = I_{\text{lighting}} \cdot I_{\text{texture}}$$

Реализация в PyOpenGL и GLFW

Для реализации используется стек технологий Python:

- **GLFW** — библиотека для создания окна, обработки ввода и управления контекстом OpenGL;
- **PyOpenGL** — Python-обёртка над OpenGL, поддерживающая как фиксированный функционал, так и шейдеры;
- **NumPy** — используется для хранения и обработки массивов данных, в том числе буфера вершин и текстур.

Реализация

В сцене отображается трёхмерный объект — тор. Его параметры определяются вручную, а поведение зависит от флагов, заданных в модуле `vars.py`.

Пример инициализации сцены:

```
1 scene_Angle = Angle(35.264, 45) # изометрическая проекция
2 scene_Scale = 0.8
3 scene_Translate = [0.0, 0.0, -3.0] # отдаление сцены
```

Построение проекции

Используется перспективная проекция. Установка осуществляется с помощью стандартных функций OpenGL, например:

```
1 glMatrixMode(GL_PROJECTION)
2 glLoadIdentity()
3 gluPerspective(60.0, width / float(height), 1.0, 20.0)
4 glMatrixMode(GL_MODELVIEW)
```

Это создаёт ощущение глубины и позволяет реалистично отображать удалённые объекты.

Анимация объекта

Анимация тора реализуется через автоматическое вращение объекта вокруг своей оси. В модуле `vars.py`:

```
1 animation = True
```

А в основном цикле программы углы изменяются на каждом кадре в зависимости от нажатий пользователя:

Функция `animate_torus()` перемещает тор и вызывает отражение от границ:

```
1 vars.My_Torus.position[i] += vars.My_Torus.velocity[i]
2 check_collision_and_reflect(i, vars.My_Torus.R + vars.My_Torus.r)
```

Это обеспечивает движения объекта в сцене.

Текстурирование тора

Наложение текстуры включается через флаг:

```
1 Torus_use_texture = True
```

При инициализации объекта загружается изображение и связывается с текстурным буфером:

```
1 texture_id = glGenTextures(1)
2 glBindTexture(GL_TEXTURE_2D, texture_id)
3 glTexImage2D(..., image_data)
4 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
```

Текстурные координаты для тора рассчитываются при генерации его вершин.

Освещение сцены

Модель освещения базируется на комбинации глобального фонового освещения (ambient), направленного света (diffuse) и отражённого света (specular).

Примеры предустановок находятся в списке:

```
1 light_presets = [
2     ([1.0, 1.0, 1.0, 0], # направление света
3     [0.1, 0.1, 0.1, 1.0], # ambient
4     [0.9, 0.9, 0.8, 1.0], # diffuse
5     [1.0, 1.0, 1.0, 1.0], # specular
6     [0.2, 0.2, 0.2, 1.0]) # глобальное ambient освещение
7 ]
```

Настройка источника света:

```
1 glLightfv(GL_LIGHT0, GL_POSITION, position)
2 glLightfv(GL_LIGHT0, GL_AMBIENT, ambient)
3 glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse)
4 glLightfv(GL_LIGHT0, GL_SPECULAR, specular)
5 glLightModelfv(GL_LIGHT_MODEL_AMBIENT, global_ambient)
```

Свойства материала объекта

Для более реалистичного освещения задаются параметры материала:

```
1 material_presets = [
2     [0.33, 0.22, 0.03, 1.0], # ambient
3     [0.78, 0.57, 0.11, 1.0], # diffuse
4     [0.99, 0.91, 0.81, 1.0], # specular
5     27.8 # shininess
6 ]
```

Применение материала:

```
1 glMaterialfv(GL_FRONT, GL_AMBIENT, ambient)
2 glMaterialfv(GL_FRONT, GL_DIFFUSE, diffuse)
3 glMaterialfv(GL_FRONT, GL_SPECULAR, specular)
4 glMaterialf(GL_FRONT, GL_SHININESS, shininess)
```

Вывод программы

Программа создала окно, отрисовала граничный куб и тор. На сцене был также представлен источник света. Тор динамически перемещался и отталкивался от границ куба. При нажатии клавиш 1-5 менялись настройки источника света. 6-9 меняли материал тора. 0 - переключала свет. Т - пеключала текстуру.

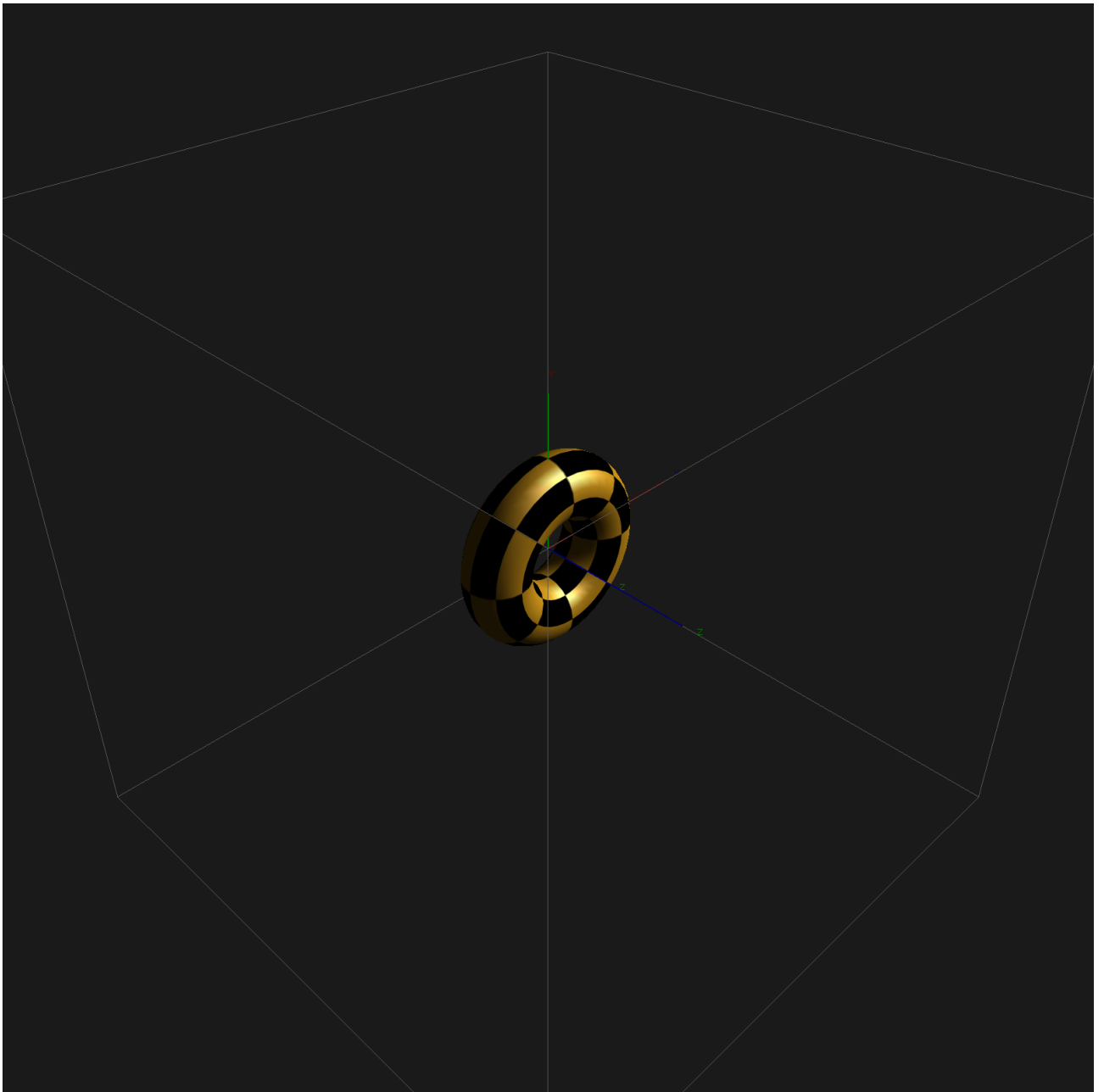


Рис. 1 — Исходная сцена

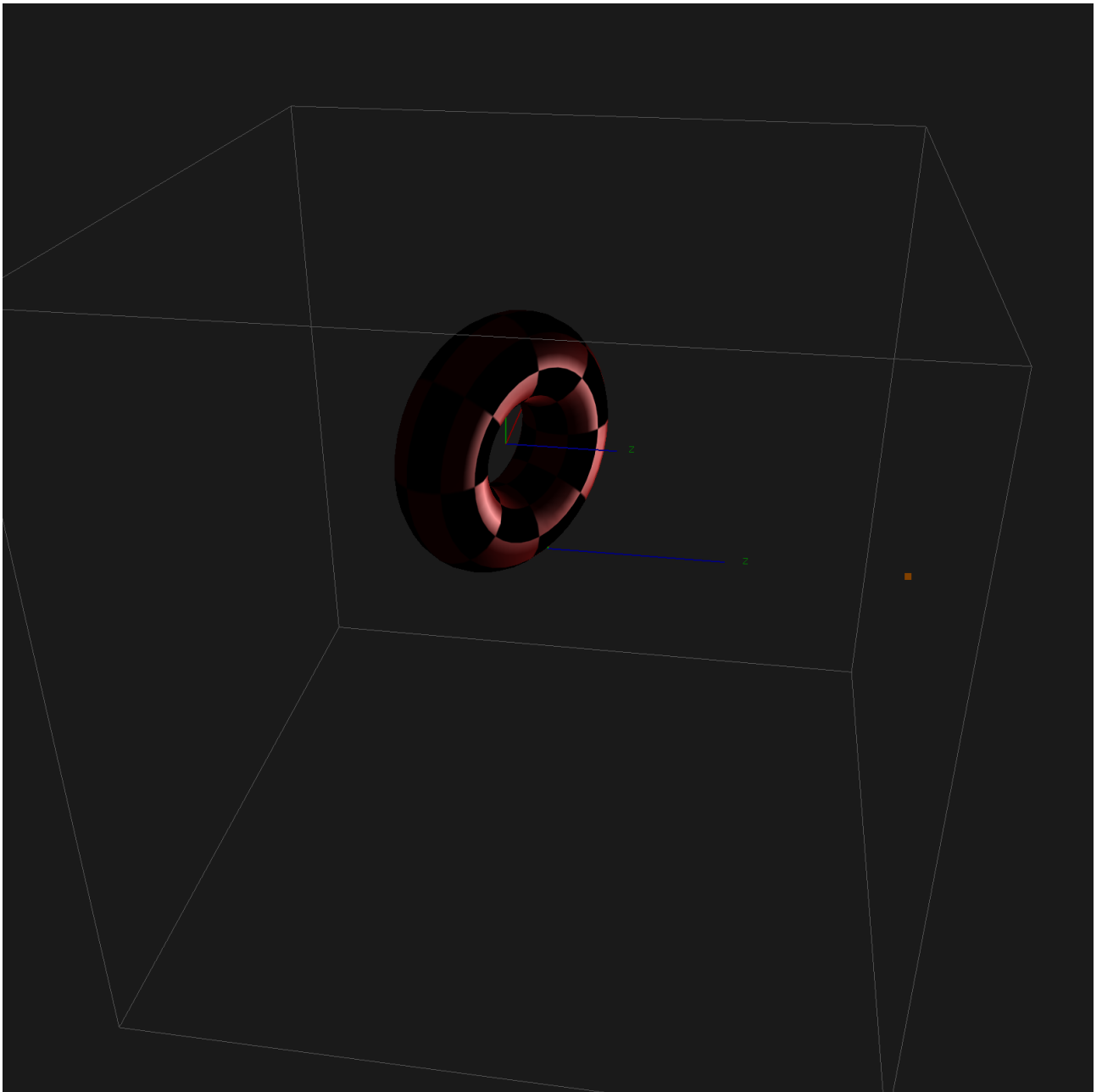


Рис. 2 — После изменения параметров

Вывод

В данной работе реализована базовая сцена с трёхмерным объектом (тором), который вращается, покрыт текстурой и освещён несколькими источниками света. Проекция выполнена с перспективой, что придаёт глубину изображению. Во время работы я вспомнил принципы работы с конвейером OpenGL, а так же узнал про модель освещения.